

R Programming for Data Analysis

Day 4 – Control Statements & Functions

Learning Objectives

By the end of Day 4, students will be able to:

- Understand decision-making in R
 - Use `if`, `else if`, and `else`
 - Use nested `if` statements
 - Use `ifelse()` for data analysis
 - Understand `for` loops
 - Understand `while` loops
 - Use `break` and `next`
 - Create custom functions
 - Pass arguments to functions
 - Return values from functions
 - Apply functions to Data Analysis problems
-

1. Introduction to Control Statements

Until now, we have written R commands that execute one after another.

For example:

```
salary <- 45000

bonus <- salary * 0.10

bonus
```

But in real-world Data Analysis, we often need to make decisions.

For example:

```
| If sales are greater than ₹50,000, classify the salesperson as "High Sales".
```

This is called **decision making**.

R provides control statements to control the flow of a program.

Main Control Statements in R

Type	Statements
Decision Making	<code>if</code> , <code>else</code> <code>if</code> , <code>else</code>
Vectorized Decision	<code>ifelse()</code>
Looping	<code>for</code> , <code>while</code>
Loop Control	<code>break</code> , <code>next</code>
Functions	<code>function()</code>

2. Comparison Operators

Control statements usually work with comparison operators.

Operator	Meaning	Example
<code>></code>	Greater than	<code>50 > 20</code>
<code><</code>	Less than	<code>20 < 50</code>
<code>>=</code>	Greater than or equal	<code>50 >= 50</code>

<code><=</code>	Less than or equal	<code>20 <= 50</code>
<code>==</code>	Equal to	<code>50 == 50</code>
<code>!=</code>	Not equal to	<code>50 != 20</code>

Example:

```
salary <- 45000

salary > 30000
```

Output:

```
TRUE
```

3. Logical Operators

Logical operators are useful when we have multiple conditions.

AND – `&`

Both conditions must be TRUE.

```
salary <- 45000
experience <- 3

salary > 30000 & experience >= 2
```

Output:

```
TRUE
```

OR – `|`

At least one condition must be TRUE.

```
salary <- 25000
experience <- 3
```

```
salary > 50000 | experience >= 2
```

Output:

```
TRUE
```

NOT – !

Reverses TRUE/FALSE.

```
x <- TRUE
```

```
!x
```

Output:

```
FALSE
```

4. if Statement

The `if` statement executes code only when a condition is TRUE.

Syntax

```
if (condition) {  
  statement  
}
```

Example

```
salary <- 45000  
  
if (salary > 40000) {  
  print("Eligible for bonus")  
}
```

Output:

```
[1] "Eligible for bonus"
```

If the salary is:

```
salary <- 30000
```

Nothing will be printed because the condition is FALSE.

5. `if` with Data Analysis Example

```
sales <- 75000

if (sales > 50000) {
  print("High Sales")
}
```

Output:

```
[1] "High Sales"
```

6. `if...else`

When we want to perform one action if the condition is TRUE and another if it is FALSE, use `else`.

Syntax

```
if (condition) {
  statement1
} else {
  statement2
}
```

Example

```
sales <- 40000

if (sales >= 50000) {
  print("Target Achieved")
} else {
  print("Target Not Achieved")
}
```

Output:

```
[1] "Target Not Achieved"
```

7. Employee Salary Example

```
salary <- 60000

if (salary >= 50000) {
  print("High Salary")
} else {
  print("Normal Salary")
}
```

8. if...else if...else

Sometimes there are multiple conditions.

Example:

- Salary $\geq$ 80000 $\rightarrow$ High
- Salary $\geq$ 50000 $\rightarrow$ Medium
- Salary $<$ 50000 $\rightarrow$ Low

```
salary <- 65000

if (salary >= 80000) {
  print("High Salary")
} else if (salary >= 50000) {
  print("Medium Salary")
} else {
  print("Low Salary")
}
```

Output:

```
[1] "Medium Salary"
```

9. Sales Performance Classification

```
sales <- 85000

if (sales >= 100000) {
  print("Excellent")
} else if (sales >= 75000) {
```

```
    print("Good")
  } else if (sales >= 50000) {
    print("Average")
  } else {
    print("Poor")
  }
}
```

Output:

```
[1] "Good"
```

10. Nested if

An `if` statement inside another `if` statement is called a **nested if**.

Example:

```
salary <- 60000
experience <- 4

if (salary >= 50000) {

  if (experience >= 3) {
    print("Eligible for promotion")
  }

}
```

Output:

```
[1] "Eligible for promotion"
```

11. ifelse() Function

`ifelse()` is extremely important in Data Analysis.

It is useful when we want to apply a condition to multiple values.

Syntax

```
ifelse(condition, value_if_true, value_if_false)
```

Example

```
salary <- c(25000, 45000, 60000, 80000)

ifelse(salary >= 50000, "High", "Low")
```

Output:

```
"Low" "Low" "High" "High"
```

12. Creating a New Data Frame Column Using `ifelse()`

```
employees <- data.frame(
  Name = c("Amit", "Rahul", "Priya", "Neha"),
  Salary = c(25000, 45000, 60000, 80000)
)

employees$Category <- ifelse(
  employees$Salary >= 50000,
  "High",
  "Low"
)

employees
```

Output conceptually:

Name	Salary	Category
Amit	25000	Low
Rahul	45000	Low
Priya	60000	High
Neha	80000	High

13. Multiple Conditions with `ifelse()`

```
employees$Salary_Level <- ifelse(
  employees$Salary >= 80000,
  "High",
  ifelse(
    employees$Salary >= 50000,
    "Medium",
    "Low"
  )
)

employees
```

This creates:

Salary	Salary_Level
25000	Low
45000	Low
60000	Medium
80000	High

14. for Loop

A `for` loop repeats a block of code for each value.

Syntax

```
for (variable in sequence) {
  statement
}
```

Example

```
for (i in 1:5) {
  print(i)
}
```

Output:

```
1
2
3
4
5
```

15. Loop Through a Vector

```
sales <- c(10000, 20000, 30000, 40000)

for (x in sales) {
  print(x)
}
```

16. Calculate Bonus Using a Loop

```
salary <- c(30000, 40000, 50000, 60000)

for (s in salary) {
  bonus <- s * 0.10
  print(bonus)
}
```

Output:

```
3000
4000
5000
6000
```

17. Loop with a Condition

```
sales <- c(20000, 60000, 45000, 90000)

for (s in sales) {

  if (s >= 50000) {
    print("Target Achieved")
  }
}
```

```
    } else {  
        print("Target Not Achieved")  
    }  
  
}
```

18. for Loop with Index

Sometimes we need the position/index of each value.

```
sales <- c(20000, 60000, 45000, 90000)  
  
for (i in 1:length(sales)) {  
  
    print(paste("Sales", i, "=", sales[i]))  
  
}
```

Output:

```
Sales 1 = 20000  
Sales 2 = 60000  
Sales 3 = 45000  
Sales 4 = 90000
```

19. while Loop

A while loop executes as long as a condition remains TRUE.

Syntax

```
while (condition) {  
    statement  
}
```

Example

```
i <- 1  
  
while (i <= 5) {  
    print(i)
```

```
i <- i + 1  
}
```

Output:

```
1  
2  
3  
4  
5
```

Important

Always make sure the condition eventually becomes FALSE.

Otherwise, you may create an **infinite loop**.

20. break

`break` stops a loop immediately.

Example:

```
for (i in 1:10) {  
  
  if (i == 6) {  
    break  
  }  
  
  print(i)  
}
```

Output:

```
1  
2  
3  
4  
5
```

21. next

`next` skips the current iteration and continues with the next one.

Example:

```
for (i in 1:5) {  
  
    if (i == 3) {  
        next  
    }  
  
    print(i)  
}
```

Output:

```
1  
2  
4  
5
```

22. Difference Between `break` and `next`

Statement	Meaning
<code>break</code>	Completely stops the loop
<code>next</code>	Skips current iteration

23. Introduction to Functions

A function is a reusable block of code that performs a specific task.

You have already used built-in functions:

```
mean()  
sum()  
max()
```

```
min()  
length()
```

Example:

```
sales <- c(10000, 20000, 30000)  
  
mean(sales)
```

Instead of repeatedly writing the same calculation, we can create our own function.

24. Creating a Custom Function

Syntax

```
function_name <- function(arguments) {  
  statements  
}
```

Example

```
calculate_bonus <- function(salary) {  
  salary * 0.10  
}
```

Now call the function:

```
calculate_bonus(50000)
```

Output:

```
5000
```

25. Function with Multiple Arguments

```
calculate_profit <- function(sales, cost) {  
  sales - cost  
}
```

Use it:

```
calculate_profit(100000, 70000)
```

Output:

```
30000
```

26. Function with `return()`

We can explicitly return a value.

```
calculate_bonus <- function(salary) {  
  
  bonus <- salary * 0.10  
  
  return(bonus)  
}
```

Use:

```
calculate_bonus(60000)
```

Output:

```
6000
```

Note

In R, the last evaluated expression is automatically returned, so `return()` is often optional.

27. Function for Sales Classification

```
sales_category <- function(sales) {  
  
  if (sales >= 100000) {  
    return("Excellent")  
  } else if (sales >= 50000) {  
    return("Good")  
  } else {  
    return("Poor")  
  }  
  
}
```

Test:

```
sales_category(120000)
```

Output:

```
"Excellent"
```

```
sales_category(70000)
```

Output:

```
"Good"
```

```
sales_category(30000)
```

Output:

```
"Poor"
```

28. Function for Employee Salary

```
calculate_annual_salary <- function(monthly_salary) {  
  
  annual_salary <- monthly_salary * 12  
  
  return(annual_salary)  
  
}
```

Use:

```
calculate_annual_salary(45000)
```

Output:

```
540000
```

29. Function with Default Argument

We can provide a default value.

```
calculate_bonus <- function(salary, rate = 0.10) {  
  
  salary * rate  
  
}
```

Use default rate:

```
calculate_bonus(50000)
```

Output:

```
5000
```

Use a different rate:

```
calculate_bonus(50000, 0.20)
```

Output:

```
10000
```

30. Functions and Vectors

Functions can also work with vectors.

```
calculate_bonus <- function(salary) {  
  salary * 0.10  
}  
  
salary <- c(30000, 40000, 50000, 60000)  
  
calculate_bonus(salary)
```

Output:

```
3000 4000 5000 6000
```

This is one of the important strengths of R: **vectorized calculations**.

31. Practical Example – Employee Data Analysis

Create the dataset:

```
employees <- data.frame(  
  Employee_ID = c(101, 102, 103, 104, 105),  
  Name = c("Amit", "Rahul", "Priya", "Neha", "Vikas"),  
  Department = c("Sales", "IT", "HR", "Sales", "IT"),  
  Salary = c(35000, 65000, 45000, 55000, 80000),  
  Experience = c(1, 5, 3, 4, 7)  
)  
  
employees
```

Add Annual Salary

```
employees$Annual_Salary <- employees$Salary * 12
```

Add Salary Category

```
employees$Salary_Category <- ifelse(  
  employees$Salary >= 60000,  
  "High",  
  "Normal"  
)
```

Add Promotion Eligibility

Suppose an employee is eligible when:

- Salary >= 50000
- Experience >= 3 years

```
employees$Promotion <- ifelse(  
  employees$Salary >= 50000 &  
  employees$Experience >= 3,  
  "Eligible",  
  "Not Eligible"  
)
```

Display:

```
employees
```

32. Practical Function – Employee Eligibility

Create a function:

```
promotion_check <- function(salary, experience) {  
  
  if (salary >= 50000 & experience >= 3) {  
    return("Eligible")  
  } else {  
    return("Not Eligible")  
  }  
  
}
```

Test:

```
promotion_check(60000, 5)
```

Output:

```
"Eligible"
```

Test:

```
promotion_check(40000, 5)
```

Output:

```
"Not Eligible"
```

33. Practical Example – Sales Data

```
sales <- data.frame(  
  Salesperson = c("Amit", "Rahul", "Priya", "Neha", "Vikas"),  
  Sales = c(45000, 80000, 35000, 120000, 65000)  
)  
  
sales
```

Create Performance Category

```
sales$Performance <- ifelse(  
  sales$Sales >= 100000,
```

```
    "Excellent",
  ifelse(
    sales$Sales >= 50000,
    "Good",
    "Needs Improvement"
  )
)

sales
```

34. Function to Calculate Discount

Suppose a customer gets:

- 20% discount if purchase $\geq$ ₹50,000
- 10% discount if purchase $\geq$ ₹20,000
- 5% otherwise

```
calculate_discount <- function(amount) {

  if (amount >= 50000) {
    return(amount * 0.20)

  } else if (amount >= 20000) {
    return(amount * 0.10)

  } else {
    return(amount * 0.05)
  }

}
```

Test:

```
calculate_discount(60000)
```

Output:

```
12000
```

35. Important Difference: `if` vs `ifelse()`

if	ifelse()
Usually works with one condition	Works efficiently with vectors
Useful for program logic	Very useful for Data Analysis
Executes a block of code	Returns values
Common inside functions	Common when creating Data Frame columns

Example:

if

```
salary <- 60000

if (salary > 50000) {
  print("High")
}
```

ifelse()

```
salary <- c(30000, 60000, 80000)
```

```
ifelse(salary > 50000, "High", "Low")
```

36. Important Difference: `for` vs `while`

<code>for</code>	<code>while</code>
Used when number of iterations is known	Used when condition controls repetition
Works naturally with vectors/sequences	Continues until condition becomes FALSE
Common for looping through data	Useful when termination depends on a condition

37. Common Mistakes

Mistake 1 – Using `=` instead of `==`

Wrong:

```
if (salary = 50000)
```

Correct:

```
if (salary == 50000)
```

`=` / `<-` are used for assignment, while `==` checks equality.

Mistake 2 – Forgetting braces

Correct:

```
if (salary > 50000) {  
  print("High")  
}
```

Mistake 3 – Infinite while loop

Wrong:

```
i <- 1  
  
while (i <= 5) {  
  print(i)  
}
```

`i` never changes.

Correct:

```
i <- 1  
  
while (i <= 5) {  
  print(i)  
  i <- i + 1  
}
```

Practice Questions – Day 4

Practice 1 – Salary Classification

Create:

```
salary <- 55000
```

Write an `if...else` statement:

- Salary $\geq$ 50000 $\rightarrow$ "High Salary"
 - Otherwise $\rightarrow$ "Normal Salary"
-

Practice 2 – Age Category

Create:

```
age <- 25
```

Classify:

- Below 18 → "Minor"
- 18–59 → "Adult"
- 60 or above → "Senior Citizen"

Use `if...else if...else`.

Practice 3 – Sales Target

Create:

```
sales <- 75000
```

Classify:

- = 100000 → Excellent
 - = 75000 → Good
 - = 50000 → Average
 - Below 50000 → Poor
-

Practice 4 – Vector Classification

Create:

```
marks <- c(35, 67, 82, 45, 91, 28)
```

Use `ifelse()` to classify:

- Marks ≥ 50 → "Pass"
 - Otherwise → "Fail"
-

Practice 5 – Multiple Conditions

Create:

```
salary <- c(25000, 45000, 60000, 85000)
```

Create a new category:

- = 75000 → High
 - = 50000 → Medium
 - Below 50000 → Low
-

Practice 6 – For Loop

Create:

```
numbers <- 1:10
```

Use a `for` loop to print every number.

Practice 7 – Even Numbers

Use a `for` loop to print only even numbers from 1 to 20.

Hint:

```
if (i %% 2 == 0)
```

Practice 8 – Calculate Bonus

Create:

```
salary <- c(30000, 40000, 50000, 60000)
```

Use a `for` loop to calculate a 10% bonus for each salary.

Practice 9 – Create a Function

Create a function:

```
calculate_area()
```

It should accept length and width and return the area of a rectangle.

Example:

```
calculate_area(10, 5)
```

Expected output:

```
50
```

Practice 10 – Profit Function

Create:

```
calculate_profit(sales, cost)
```

The function should return:

```
sales - cost
```

Example:

```
calculate_profit(100000, 65000)
```

Expected output:

```
35000
```



Day 4 Mini Project – Employee Performance

Analysis

Create the following Data Frame:

```
employees <- data.frame(  
  Name = c("Amit", "Rahul", "Priya", "Neha", "Vikas", "Sneha"),  
  Department = c("Sales", "IT", "Sales", "HR", "IT", "Sales"),  
  Salary = c(35000, 70000, 55000, 45000, 85000, 65000),  
  Sales = c(45000, 30000, 75000, 25000, 50000, 110000),  
  Experience = c(2, 6, 4, 3, 8, 5)  
)
```

Perform the following:

Task 1

Display the Data Frame.

Task 2

Create:

```
Annual_Salary
```

using monthly salary $\times$ 12.

Task 3

Create:

```
Salary_Category
```

Rules:

- = 75000 → High
- = 50000 → Medium
- Below 50000 → Low

Task 4

Create:

```
Sales_Category
```

Rules:

- = 100000 → Excellent
- = 75000 → Good
- = 50000 → Average
- Below 50000 → Poor

Task 5

Create:

```
Promotion
```

An employee is eligible if:

```
Salary >= 50000  
AND  
Experience >= 3
```

Otherwise:

```
Not Eligible
```

Task 6

Create a function:

```
calculate_bonus <- function(salary) {  
  salary * 0.10  
}
```

```
}
```

Use it to create:

```
Bonus
```

column.

Task 7

Display employees who are eligible for promotion.

Task 8

Display employees whose sales are above ₹50,000.

Task 9

Find the employee with the highest sales.

Task 10

Find the average salary.



Day 4 Quick Revision

Concept	Syntax
If	<pre>if(condition)</pre>
If Else	<pre>if() {} else {}</pre>
Multiple Conditions	<pre>if() {} else if() {} else {}</pre>
Vectorized Condition	<pre>ifelse()</pre>
For Loop	<pre>for(i in</pre>

	<code>sequence)</code>
While Loop	<code>while(c ondition)</code>
Stop Loop	<code>break</code>
Skip Iteration	<code>next</code>
Function	<code>functio n()</code>
Return Value	<code>return()</code>

★ Important Concepts to Remember

1. Decision Making

```
if()
else if()
else
```

2. Data Analysis Conditions

```
ifelse()
```

3. Repetition

```
for()
while()
```

4. Loop Control

```
break
next
```

5. Reusable Code

```
function()
```

Day 4 Learning Outcome

After completing Day 4, students should be able to take a Data Frame and build simple analytical logic such as:

```
Raw Data
  ↓
Condition
  ↓
Classification
  ↓
Calculation
  ↓
New Column
  ↓
Analysis
```

For example:

```
employees$Performance <- ifelse(
  employees$Sales >= 50000,
  "Good",
  "Needs Improvement"
)
```

This forms the foundation for **data cleaning, transformation and automation**, which will be used heavily in the upcoming `dplyr` lessons.